

1. Objective :

Diabetes poses a significant health concern worldwide due to its prevalence and impact on quality of life. Early detection plays a crucial role in effective disease management and prevention of severe complications. This project seeks to leverage machine learning methodologies, specifically a Support Vector Machine (SVM) model, to accurately predict the presence or absence of diabetes using accessible clinical variables. By building a robust diabetes prediction system, the project aims to empower both individuals and healthcare professionals with rapid, data-driven risk assessments.

2. Topic Description :

- Diabetes Prediction Using Machine Learning Algorithms.
- This project focuses on the application of machine learning techniques to predict the likelihood of diabetes in individuals using easily accessible clinical and biometric variables. Diabetes, a chronic metabolic disorder, affects millions globally and is associated with severe health complications if not identified and managed early. Traditional diagnostic methods can be resource-intensive and are sometimes inaccessible to at-risk populations.
- By leveraging well-established algorithm Random Forest, this work demonstrates a robust, data-driven approach for early diabetes detection. The project utilizes the Pima Indians Diabetes Database—a widely recognized benchmark dataset in biomedical research, containing factors such as age, pregnancies, glucose level, blood pressure, skin thickness, insulin, BMI, diabetes pedigree function, and outcome (diabetic or non-diabetic).
- The core objective is to build and evaluate predictive models that can accurately classify individuals as diabetic or non-diabetic. The ultimate goal is to support healthcare professionals and the general public with rapid, reliable risk assessment tools, thereby facilitating early intervention, personalized health planning, and improved quality of life. Through comprehensive data exploration, model development, and rigorous evaluation, the project illustrates the impact and potential of modern machine learning in medical diagnostics.

3. Methodology :

Data Exploration & Preparation

- Dataset: Utilized the Pima Indians Diabetes dataset, containing 768 records and 9 columns (including the binary outcome label).
- Preprocessing Approaches:
 - Loaded data using pandas.
 - Explored the distribution and correlation of features to inform preprocessing decisions.
 - No manual feature mapping required as labels are provided as binary (0 for healthy, 1 for diabetic).
 - Randomly shuffled the dataset to remove ordering bias.
- Splitting: Dataset divided into training and testing subsets for unbiased evaluation.

Feature Analysis :

- Explored the concentration and spread of key features such as glucose, blood pressure, BMI, and diabetes pedigree function.
- Noted that most values are concentrated in plausible physiological ranges, with the “Diabetes Pedigree Function” centered below 0.63 and averaging around 0.47.

Model Construction

Random Forest

Random Forest is an ensemble learning algorithm that builds a "forest" of decision trees and makes final predictions via majority vote (classification) or averaging (regression).

- Process:
 - Each tree is trained on a random sample of the data and considers a random subset of features when splitting nodes.
 - This randomness results in less susceptible models to overfitting and greater prediction robustness.
- Advantages:

- Handles non-linearity and complex feature interactions effectively.
- Provides feature importance metrics.
- Robust to noise and missing values.
- Training:
 - Fitted to the same feature set as the SVM for comparative analysis.

4.Code :

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('diabetes.csv')

# Prepare features and target
X = df.drop(['Outcome'], axis=1)
y = df['Outcome']

# Split dataset
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)

# Train SVM model
svm = SVC(kernel='linear') # Alternatives: 'rbf', 'poly', etc.
svm.fit(x_train, y_train)

# Function to predict based on user input
def predict_diabetes(user_input_dict):
    user_data = pd.DataFrame(user_input_dict, index=[0])
    prediction = svm.predict(user_data)[0]
    accuracy = accuracy_score(y_test, svm.predict(x_test))
    return prediction, accuracy

# Get user input
print("\nEnter the following health information:\n")
user_input = {
    'Pregnancies': float(input("Pregnancies (e.g. 2): ")),
    'Glucose': float(input("Glucose (0-200): ")),
    'BloodPressure': float(input("Blood Pressure (mmHg): ")),
```

```

'SkinThickness': float(input("Skin Thickness (0-100): ")),
'Insulin': float(input("Insulin (0-846): ")),
'BMI': float(input("BMI (Body Mass Index): ")),
'DiabetesPedigreeFunction': float(input("Diabetes Pedigree Function
(e.g. 0.47): ")),
'Age': float(input("Age: "))
}
# Run prediction
result, acc = predict_diabetes(user_input)
# Show result
print("\n🔍 Prediction Result:")
if result == 1:
    print("⚠️ You might have **Diabetes**.")
else:
    print("✅ You are likely **Healthy**.")
print(f"📊 Model Accuracy: {acc * 100:.2f}%")

```

5.Code outputs :

Enter the following health information:

Pregnancies (e.g. 2): 3
 Glucose (0-200): 126
 Blood Pressure (mmHg): 88
 Skin Thickness (0-100): 41
 Insulin (0-846): 235
 BMI (Body Mass Index): 39.3
 Diabetes Pedigree Function (e.g. 0.47): .704
 Age: 27

🔍 Prediction Result:
 ✅ You are likely **Healthy**.
 📊 Model Accuracy: 81.17%

Enter the following health information:

Pregnancies (e.g. 2): 0
 Glucose (0-200): 118
 Blood Pressure (mmHg): 84
 Skin Thickness (0-100): 47
 Insulin (0-846): 230
 BMI (Body Mass Index): 45.8
 Diabetes Pedigree Function (e.g. 0.47): 0.551
 Age: 31

🔍 Prediction Result:
 ⚠️ You might have **Diabetes**.
 📊 Model Accuracy: 81.82%

Enter the following health information:

Pregnancies (e.g. 2): 10
 Glucose (0-200): 139
 Blood Pressure (mmHg): 80
 Skin Thickness (0-100): 0
 Insulin (0-846): 0
 BMI (Body Mass Index): 27.1
 Diabetes Pedigree Function (e.g. 0.47): 1.441
 Age: 57

🔍 Prediction Result:
 ✅ You are likely **Healthy**.
 📊 Model Accuracy: 83.12%

6.Code Explanation :

Import Required Libraries :

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Load Dataset :

Loads the diabetes dataset from a CSV file into a pandas DataFrame.

```
df = pd.read_csv('diabetes.csv')
```

Set Features and Target :

x: Contains all the input features (all columns except 'Outcome').

y: Contains the labels: 1 (diabetic) or 0 (healthy).

```
X = df.drop(['Outcome'], axis=1)
y = df['Outcome']
```

Split Data into Training and Testing Sets :

Splits the data into 80% for training and 20% for testing . random_state = 0 ensures results are reproducible .

```
x_train, x_test, y_train,
y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Train the Random Forest Model :

Initializes and trains a Random Forest model using the training data.

```
rf = RandomForestClassifier()
rf.fit(x_train, y_train)
```

Define User Prediction Function :

```
def predict_diabetes(user_input_dict):  
    user_data = pd.DataFrame(user_input_dict, index=[0])  
    prediction = rf.predict(user_data)[0]  
    accuracy = accuracy_score(y_test, rf.predict(x_test))  
    return prediction, accuracy
```

Collect User Input :

Prompts the user to enter their health-related information for all features required by the model.

Inputs are converted to the correct numeric type

```
user_input = {  
    'Pregnancies': float(input("Pregnancies (e.g. 2): ")),  
    'Glucose': float(input("Glucose (0-200): ")),  
    'BloodPressure': float(input("Blood Pressure (mmHg): ")),  
    'SkinThickness': float(input("Skin Thickness (0-100): ")),  
    'Insulin': float(input("Insulin (0-846): ")),  
    'BMI': float(input("BMI (Body Mass Index): ")),  
    'DiabetesPedigreeFunction': float(input("Diabetes Pedigree Function(0.47):")),  
    'Age': float(input("Age: "))  
}
```

Make Prediction and Show Results :

```
# Run prediction  
result, acc = predict_diabetes(user_input)  
# Show result  
print("\n💡 Prediction Result:")  
if result == 1:  
    print("⚠️ You might have **Diabetes**.")  
else:  
    print("✅ You are likely **Healthy**.")  
  
print(f"📊 Model Accuracy: {acc * 100:.2f}%")
```

7. Conclusion

This project applies the Random Forest algorithm for diabetes prediction using easily obtainable clinical data. The model exhibits solid accuracy, making it suitable for initial risk screening and education. Random Forest's ability to highlight key factors contributing to predictions makes it not only effective but also insightful for users and practitioners. However, model predictions should complement—not replace—professional medical evaluation. For future improvements, ongoing validation with diverse real-world data is recommended.

References:

- **Diabetes Dataset:** <https://www.kaggle.com/datasets/akshaydattatraykhare/diabetes-dataset>
- **scikit-learn documentation:** <https://scikit-learn.org/stable/documentation.html>